Feature Engineering

Transformation • Construction • Selection • Extraction

Definitions + examples + Google Colab code

Transform | Construct | Select | Extract

Data Science Journey • Part 3 →

What is Feature Engineering?

The art of making data model-ready

Creating, changing and choosing the input variables (features) so a model can learn better from the data.

Better accuracy	good features beat fancy algorithms
Faster training	fewer, cleaner columns
Less overfitting	remove noise & redundancy
More insight	features carry business meaning

"Garbage in, garbage out"

The 4 Pillars

Know which one you need

1. Transformation

change form / scale / distribution of existing features

e.g. scale, log, encode, bin

2. Construction

create NEW features from existing ones

e.g. family_size = sibsp + parch + 1

3. Selection

keep only the most useful features

e.g. drop noisy / redundant columns

4. Extraction

compress many features into fewer new ones

e.g. PCA: 20 columns → 3 components

1. Feature Transformation

DEFINITION: changing the scale, shape or format of existing features without creating new information.

Scaling / Normalization

Standard, Min-Max, Robust scaler

e.g. age 22 → -0.53 (z-score)

Mathematical transforms

log, sqrt, Box-Cox, Yeo-Johnson

e.g. fare 7.25 → log1p = 2.11

Encoding categorical

One-Hot, Label, Ordinal, Target

e.g. sex: male/female → 0/1

Binning / Discretization

equal-width, quantile, custom bins

e.g. age 22 → 'young adult'

Missing & outlier handling

mean/median impute, clip, winsorize

e.g. age NaN → class median

Transformation: Code

Google Colab • Titanic dataset

```
import pandas as pd, numpy as np, seaborn as sns
from sklearn.preprocessing import (StandardScaler,
    MinMaxScaler, RobustScaler, PowerTransformer,
    KBinsDiscretizer)

df = sns.load_dataset("titanic")
df["age"] = df["age"].fillna(df["age"].median())

# 1. Scaling
df["age_std"] = StandardScaler().fit_transform(df[["age"]])
df["age_mm"]  = MinMaxScaler().fit_transform(df[["age"]])
df["fare_rob"]= RobustScaler().fit_transform(df[["fare"]])

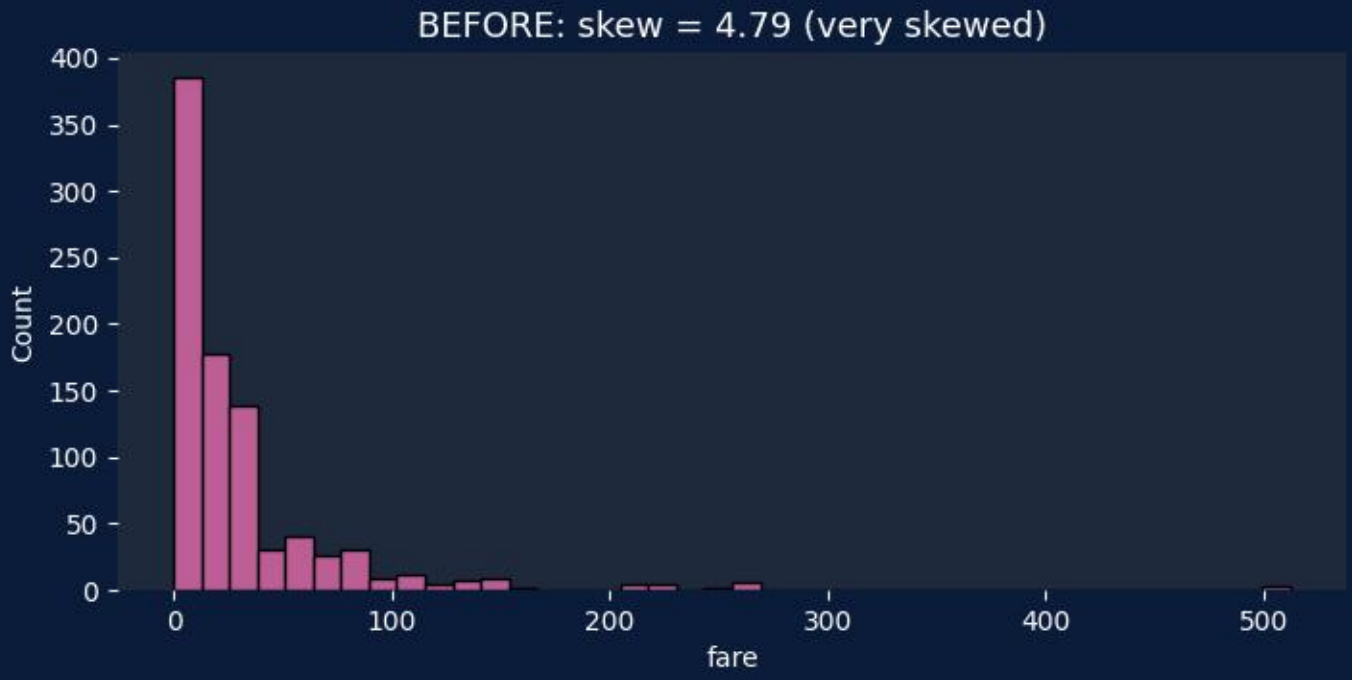
# 2. Fix skewness
df["fare_log"] = np.log1p(df["fare"])
df["fare_yj"]  = PowerTransformer().fit_transform(df[["fare"]])
print(df[["fare","fare_log"]].skew())

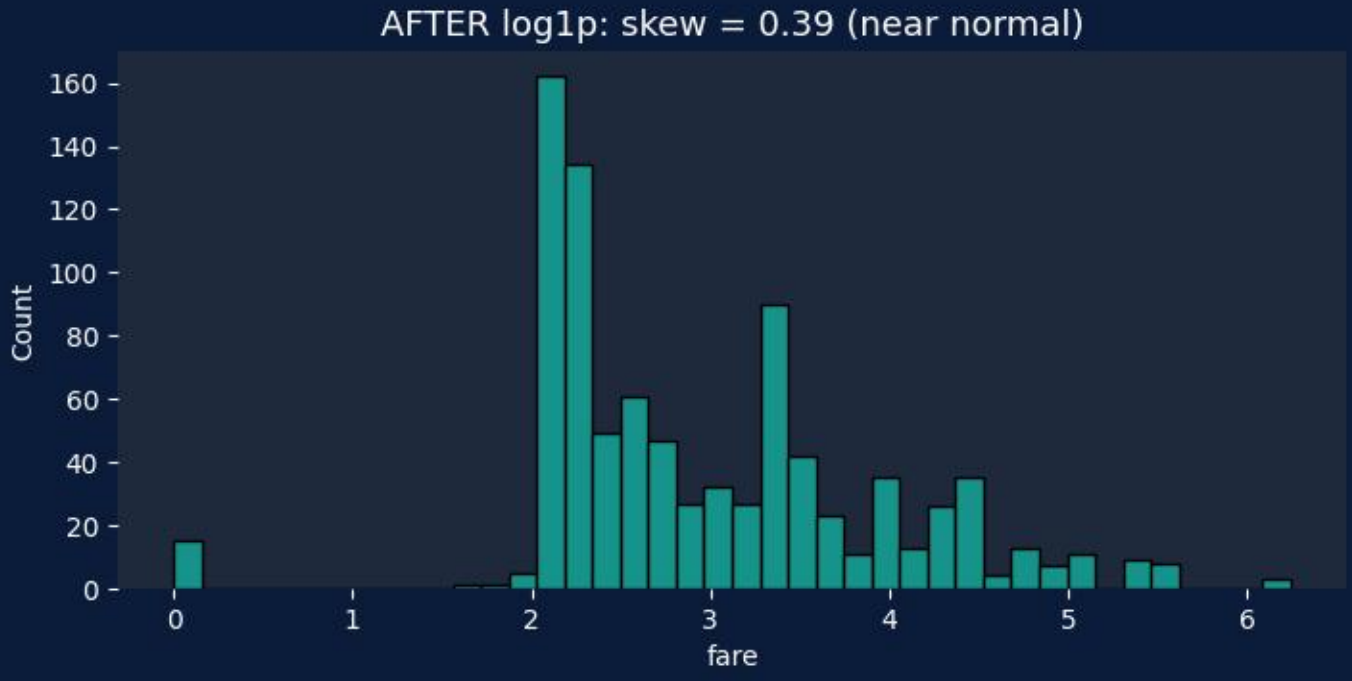
# 3. Encoding
df = pd.get_dummies(df, columns=["sex","embarked"],
                    drop_first=True)
df["class_ord"] = df["class"].map(
    {"Third":1, "Second":2, "First":3})

# 4. Binning
df["age_bin"] = pd.cut(df["age"], [0,12,18,60,100],
    labels=["child","teen","adult","senior"])
df["age_kb"] = KBinsDiscretizer(5, encode="ordinal",
    strategy="quantile").fit_transform(df[["age"]])
```

Transformation in Action

Titanic fare before vs after log transform





2. Feature Construction

DEFINITION: creating NEW features from existing data using domain knowledge, maths or aggregation.

Arithmetic / Domain

add, subtract, combine columns

e.g. family_size = sibsp + parch + 1

Ratio features

divide one column by another

e.g. fare_per_person = fare / family_size

Interaction features

multiply / combine two features

e.g. age × pclass, sex_pclass

Polynomial features

squares, cubes, cross terms

e.g. age², fare², age×fare

Date-time features

extract parts from timestamps

e.g. month, weekday, is_weekend

Aggregation (group-by)

mean/count per group

e.g. avg_fare_by_class

Construction: Code

Google Colab • Titanic dataset

```
import pandas as pd, seaborn as sns
from sklearn.preprocessing import PolynomialFeatures

df = sns.load_dataset("titanic")

# 1. Domain / arithmetic
df["family_size"] = df["sibsp"] + df["parch"] + 1
df["is_alone"] = (df["family_size"] == 1).astype(int)

# 2. Ratio
df["fare_per_person"] = df["fare"] / df["family_size"]

# 3. Interaction
df["age_x_class"] = df["age"] * df["pclass"]
df["sex_class"] = df["sex"] + "_" + df["pclass"].astype(str)

# 4. Polynomial
poly = PolynomialFeatures(degree=2, include_bias=False)
P = poly.fit_transform(df[["age","fare"]].fillna(0))
print(poly.get_feature_names_out())

# 5. Date-time
d = pd.DataFrame({"date": pd.date_range("2026-01-01",
                                        periods=10)})
d["month"]   = d["date"].dt.month
d["weekday"] = d["date"].dt.dayofweek
d["is_weekend"] = (d["weekday"] >= 5).astype(int)

# 6. Aggregation
df["avg_fare_class"] = df.groupby("pclass")["fare"] \
                         .transform("mean")
```

3. Feature Selection

DEFINITION: choosing the most relevant subset of existing features and dropping the noisy, irrelevant or redundant ones.

Filter methods (fast)

variance threshold, correlation, chi-square, ANOVA F-test, mutual information

e.g. drop features with corr > 0.9

Wrapper methods (accurate)

forward selection, backward elimination, RFE (recursive feature elimination)

e.g. RFE keeps the best 5 features

Embedded methods

built into training: Lasso (L1), Ridge, tree-based importance

e.g. Lasso shrinks weak coefs to 0

Selection: Code

Google Colab • Titanic dataset

```
import pandas as pd, seaborn as sns
from sklearn.feature_selection import (VarianceThreshold,
    SelectKBest, f_classif, mutual_info_classif, RFE)
from sklearn.linear_model import LogisticRegression, LassoCV
from sklearn.ensemble import RandomForestClassifier

df = sns.load_dataset("titanic")
cols = ["pclass","sex","age","sibsp","parch","fare","embarked"]
X = pd.get_dummies(df[cols], drop_first=True).astype(float)
X["age"] = X["age"].fillna(X["age"].median())
y = df["survived"]

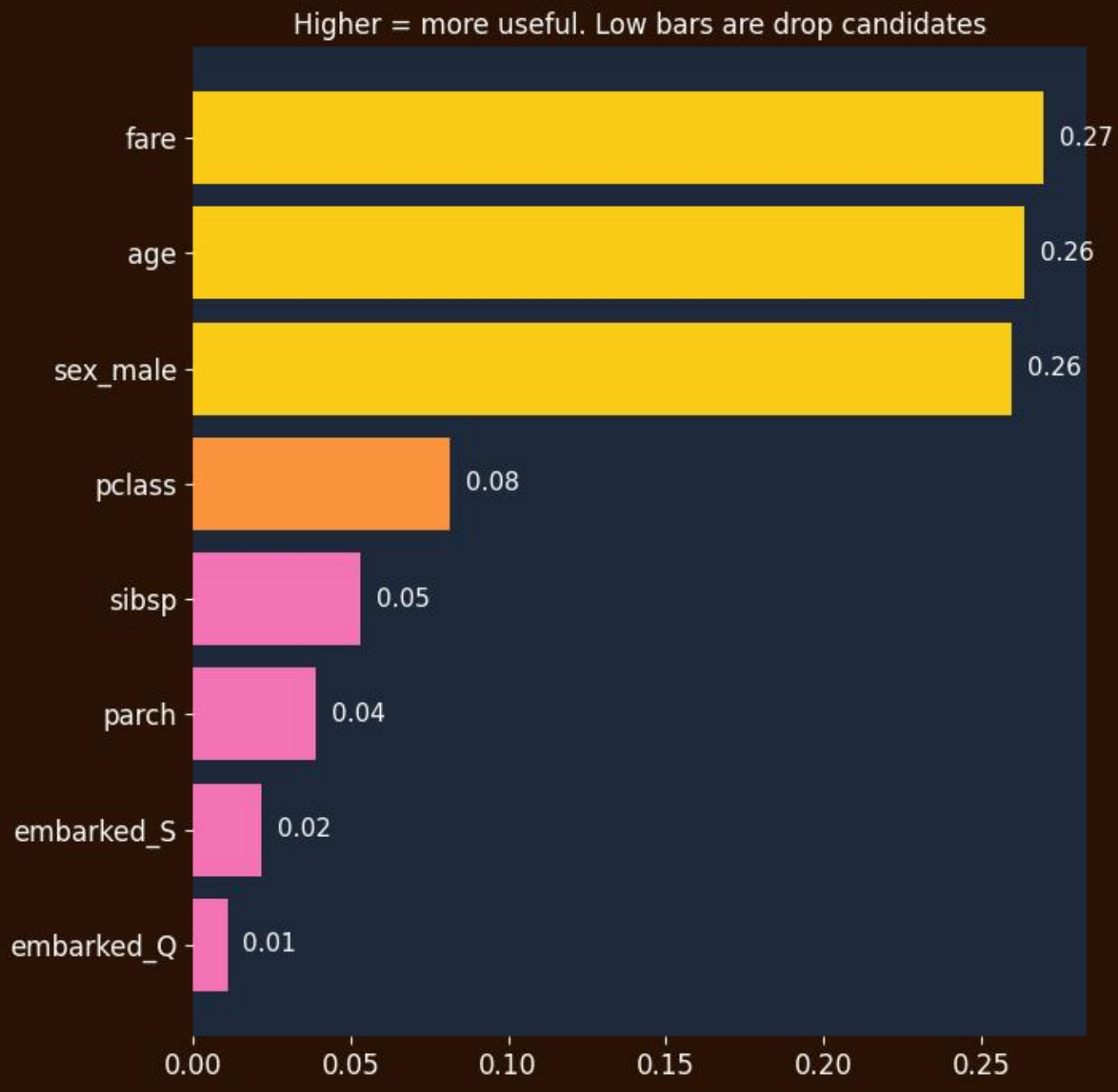
# FILTER
vt = VarianceThreshold(0.01).fit(X)
skb = SelectKBest(f_classif, k=5).fit(X, y)
print(X.columns[skb.get_support()].tolist())
print(pd.Series(mutual_info_classif(X, y),
                index=X.columns).sort_values())

# WRAPPER (RFE)
rfe = RFE(LogisticRegression(max_iter=1000),
          n_features_to_select=5).fit(X, y)
print(X.columns[rfe.support_].tolist())

# EMBEDDED
rf = RandomForestClassifier(random_state=42).fit(X, y)
print(pd.Series(rf.feature_importances_,
      index=X.columns).sort_values(ascending=False))
lasso = LassoCV(cv=5).fit(X, y)
print(dict(zip(X.columns, lasso.coef_.round(3))))
```

Selection in Action

Random Forest feature importance (Titanic)



4. Feature Extraction

DEFINITION: transforming raw or high-dimensional data into a smaller set of NEW features that keep most of the information.

PCA (unsupervised, linear)

finds directions of max variance

e.g. 20 columns → 3 components

LDA (supervised, linear)

maximises class separation

e.g. project to best class-separating axis

t-SNE / UMAP (non-linear)

preserve neighbourhoods, for visualisation

e.g. 2D map of 784-pixel images

Autoencoders (deep learning)

neural net compresses to a bottleneck

e.g. image → 32-number embedding

Text / Image features

TF-IDF, word embeddings, CNN features

e.g. review text → TF-IDF vector

Extraction: Code

Google Colab • Titanic + text

```
import pandas as pd, seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import \
    LinearDiscriminantAnalysis as LDA
from sklearn.manifold import TSNE
from sklearn.feature_extraction.text import TfidfVectorizer

df = sns.load_dataset("titanic")
cols = ["pclass","sex","age","sibsp","parch","fare","embarked"]
X = pd.get_dummies(df[cols], drop_first=True).astype(float)
X["age"] = X["age"].fillna(X["age"].median())
y = df["survived"]
Xs = StandardScaler().fit_transform(X)      # scale first!

# PCA: keep 95% of the variance
pca = PCA(n_components=0.95)
Xp = pca.fit_transform(Xs)
print(Xp.shape, pca.explained_variance_ratio_.round(2))

# LDA (supervised)
Xl = LDA(n_components=1).fit_transform(Xs, y)

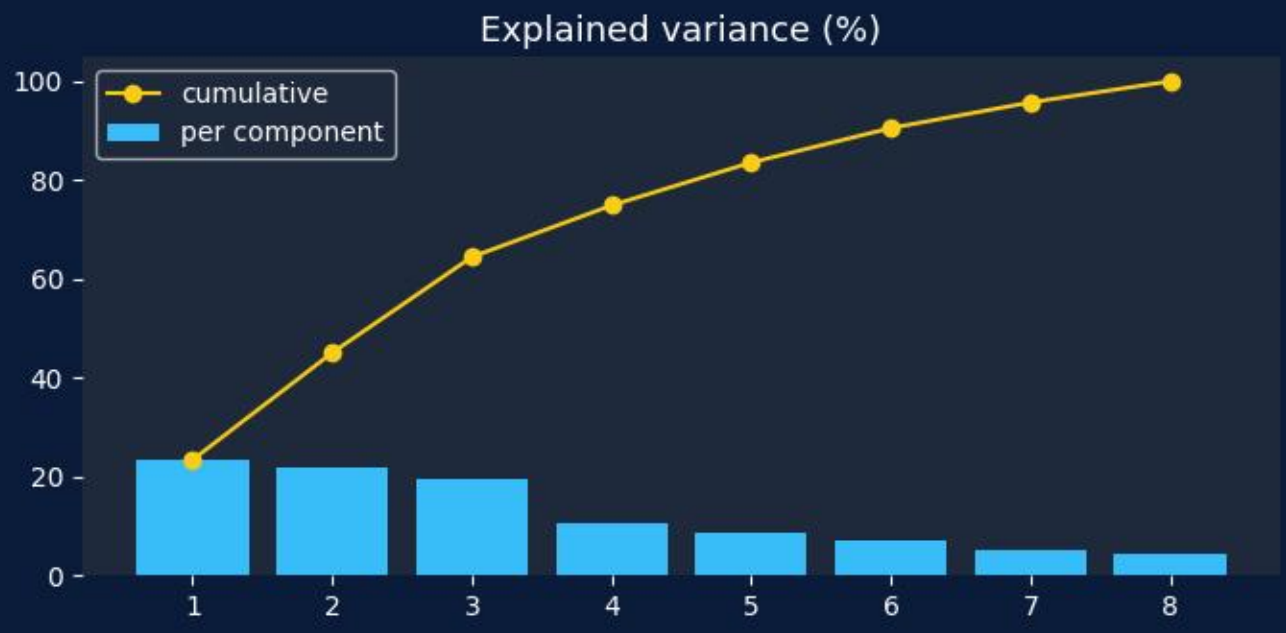
# t-SNE (2D visualisation)
Xt = TSNE(n_components=2, random_state=42).fit_transform(Xs)

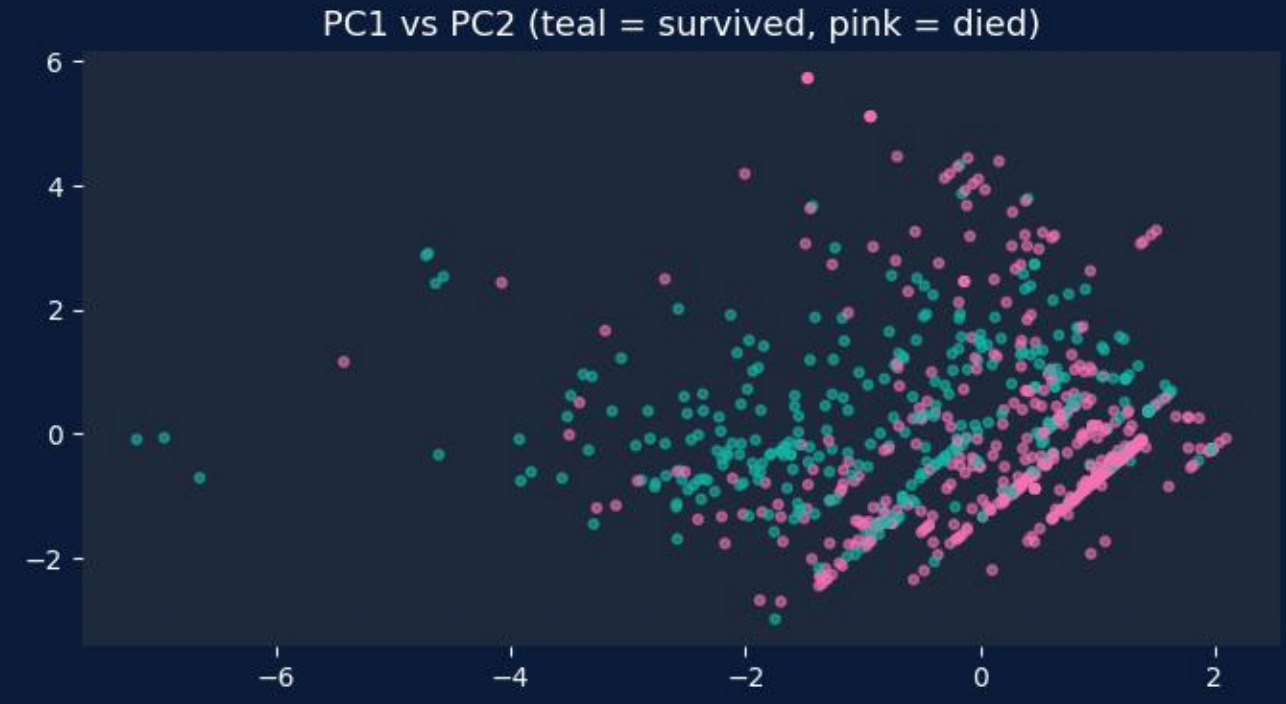
# Text: TF-IDF
docs = ["great phone camera", "battery life is poor",
        "great battery and camera"]
tfidf = TfidfVectorizer()
T = tfidf.fit_transform(docs)
print(tfidf.get_feature_names_out())

# Deep: !pip install umap-learn  (UMAP)
# Autoencoder: Keras Dense encoder -> bottleneck
```

Extraction in Action

PCA on Titanic: 8 features → components





Cheat Sheet

Which one should I use?

Skewed numbers, mixed scales → **Transformation**

Need richer signal / domain logic → **Construction**

Too many columns, noisy data → **Selection**

High dimensions, text, images → **Extraction**

Selection vs Extraction:
Select = keep a SUBSET of old features
Extract = build NEW combined features

Found this useful?

Like

Repost

Follow for the next topic: Model Building

#DataScience #FeatureEngineering #Python #MachineLearning #Colab